

VIETNAM NATIONAL UNIVERSITY, HANOI
HANOI SCHOOL OF BUSINESS AND MANAGEMENT

FINAL PROJECT REPORT
HSB2006 - DEVELOPING BUSINESS APPLICATIONS

OFFICESTOCK

Office Stationery Inventory and Internal Issuance Management System

Project Information	Value
Semester / Academic Year	Semester 2 - Academic Year 2024-2025
Class	MET4
Project	OfficeStock
Production Architecture	Vercel Frontend -> PHP API -> Online MySQL/MariaDB
Roles	ADMIN_MANAGER / WAREHOUSE / EMPLOYEE
Team members	Đăng - Trung - Huyền - Duy - Vinh
GitHub Repository	[TO BE UPDATED - ACTUAL GITHUB URL]
Project Board	[TO BE UPDATED - ACTUAL PROJECT BOARD URL]
Live Frontend	[TO BE UPDATED - ACTUAL VERCEL URL]
Backend API	[TO BE UPDATED - ACTUAL PHP API URL]
Database Provider	[TO BE UPDATED - ACTUAL ONLINE DATABASE PROVIDER]

Important evidence rule: public URLs, screenshots and PASS/FAIL results must be inserted only after real production deployment and execution.

Report Content

- 1. Executive Summary
- 2. Business Problem
- 3. Project Scope, Users, Assumptions and Limitations
- 4. Objectives
- 5. Functional and Non-functional Requirements
- 6. User Stories and Acceptance Criteria
- 7. Project Management, Project Board and Team Contribution
- 8. System Diagrams
- 9. Application Architecture and Technology Stack
- 10. Database Schema and Data Dictionary
- 11. Implementation Evidence
- 12. UI/UX Design Workflow
- 13. Authentication and Authorization
- 14. Core Business Workflow
- 15. Search, Filter, Sort and Reporting
- 16. Validation and Error Handling
- 17. Security Controls
- 18. Testing, Results and Defects
- 19. Deployment and Setup Guide
- 20. Test Accounts and Demo Scenario
- 21. Requirement-to-Evidence Traceability Matrix
- 22. Third-party Assets and Licenses
- 23. AI/Tool-use Declaration
- 24. Limitations and Future Development
- 25. Viva / Demo Preparation
- 26. Final Submission Checklist

1. Executive Summary

OfficeStock is a database-driven business web application that manages office-supply master data, inventory quantities, internal stationery requests, managerial approval and warehouse issuance. The production architecture separates the public frontend from the PHP business backend: a responsive frontend is prepared for Vercel, a PHP 8.x JSON API contains authentication, authorization and business rules, and all operational data is stored in an online MySQL/MariaDB database.

The final academic scope intentionally contains exactly three roles: ADMIN_MANAGER, WAREHOUSE and EMPLOYEE. The central demonstrable workflow is Employee Request -> Manager Approval -> Warehouse Issue -> Automatic Inventory Update -> Transaction History. Warehouse issuance uses a database transaction and row-level locking so an insufficient item causes the entire issuance to roll back rather than partially deduct stock.

The source package has been refactored and statically verified for PHP syntax, JavaScript syntax and the frontend production build. Public hosting URLs and production runtime test results remain evidence items that must be supplied by the team after connecting real hosting/database accounts. This report therefore does not invent production URLs, screenshots or PASS results.

2. Business Problem

- Office-supply quantities can become inconsistent when receipts and issues are recorded in separate spreadsheets or messages.
- Employees may not have a standardized channel to submit and track stationery requests.
- Approval and warehouse issuance can be difficult to trace when responsibility is not recorded in one system.
- Low-stock products may be noticed late, increasing the risk of unavailable everyday office supplies.
- Transaction reporting becomes time-consuming when records are fragmented.
- Different user responsibilities require server-side access control rather than a shared unrestricted interface.

2.1 Product Idea

OfficeStock provides a focused internal workflow rather than a large ERP scope. It connects product/supplier master data, inventory, employee request, manager approval, warehouse issue and reporting in one database-driven application.

2.2 Business Value

- Improved traceability of who requested, reviewed and issued stationery.
- Consistent current-stock visibility after every inventory-changing transaction.
- A clear status lifecycle: PENDING, APPROVED/REJECTED and ISSUED.
- Low-stock visibility based on quantity compared with minimum_stock.
- A reporting source based on inventory transaction records rather than manually consolidated files.

3. Project Scope, Users, Assumptions and Limitations

3.1 In Scope / Out of Scope

In Scope	Out of Current MVP
Authentication + exactly 3 roles	ERP / accounting
Product CRUD	Payment gateway
Supplier CRUD	CRM
Inventory + low-stock detection	AI demand forecasting
Stock In / Stock Out	Multi-company tenancy

Employee multi-item request	Multi-warehouse transfer
Manager approve/reject	Barcode hardware
Warehouse issue + automatic stock update	Email/SMS production notifications
Search / Filter / Sort / Reporting	Commercial SLA / high-availability design
Production deployment-ready architecture	Native mobile application

3.2 Users and Responsibilities

Role	Responsibilities
ADMIN_MANAGER	Dashboard; Product/Supplier CRUD; inventory view; request approve/reject; reports; users; transaction history.
WAREHOUSE	Dashboard; product/inventory view; Stock In/Out; view approved requests; issue request; transaction history.
EMPLOYEE	Dashboard; active stationery catalog; create multi-item request; view/cancel own PENDING request.

3.3 Assumptions

- The academic MVP represents one organization and one logical warehouse.
- Inventory quantities are stored as non-negative integers.
- Each employee belongs to a department so issued requests can be attributed to that department.
- The assessment database contains non-sensitive sample data and dedicated test accounts.
- The public frontend, PHP backend and database are separate production resources connected over the Internet.

3.4 Limitations

- No MFA/SSO.
- No automatic procurement/replenishment order.
- No multi-warehouse transfer.
- No production notification service.
- Backups, monitoring and high availability depend on the selected hosting/database provider.

4. Project Objectives

- Translate an office-supply management problem into a secure database-driven business web application.
- Digitize the request -> approval -> issue process.
- Maintain consistent stock quantity after stock in, stock out and approved-request issuance.
- Provide role-appropriate interfaces for manager, warehouse and employee users.
- Provide search/filter/sort/reporting features using live database data.
- Prepare a production architecture with a public frontend, public PHP API and online relational database.
- Provide evidence that is explainable and verifiable during demonstration/viva.

5. Functional and Non-functional Requirements

ID	Functional Requirement
FR01	Authenticate users and revoke application sessions on logout.
FR02	Implement exactly three server-side roles.

FR03	Provide database-derived role dashboards.
FR04	Product create/read/update/deactivate/restore.
FR05	Supplier create/read/update/deactivate/restore.
FR06	Inventory search/filter/sort and low-stock detection.
FR07	Warehouse multi-item Stock In.
FR08	Warehouse direct Stock Out with negative-stock prevention.
FR09	Employee creates a multi-item stationery request.
FR10	Employee views and cancels only own PENDING requests.
FR11	Admin/Manager approves or rejects PENDING requests.
FR12	Warehouse issues only APPROVED requests.
FR13	Issue transaction atomically reduces inventory and changes request to ISSUED.
FR14	Transaction history is available to authorized operational roles.
FR15	Manager report filters by transaction type/date and returns a database summary.
FR16	Manager can create and lock/unlock assessment users.

ID	Non-functional Requirement
NFR01	Responsive pages with consistent navigation, forms, validation messages and status feedback.
NFR02	PHP 8.x backend with PDO connection to MySQL/MariaDB.
NFR03	Relational schema with PK, FK, unique constraints and transaction integrity.
NFR04	Prepared statements for user-controlled query values.
NFR05	Hashed passwords and protected application sessions.
NFR06	Server-side authorization on privileged endpoints.
NFR07	Secrets supplied using environment variables and excluded from source control.
NFR08	Frontend prepared for Vercel and backend prepared for Docker web hosting.
NFR09	No static/fake business data used to simulate backend features.
NFR10	Production test results must be based on actual execution.

6. User Stories and Acceptance Criteria

ID	User Story	Acceptance Criteria
US01	As an Employee, I want to sign in so that I can access my own stationery workflow.	Given valid Employee credentials, when I sign in, then the Employee dashboard is displayed and manager/warehouse actions are not authorized.
US02	As an Employee, I want to request multiple stationery items in one request.	A request requires a reason and at least one unique product with quantity > 0; successful submission creates PENDING header/detail

		records.
US03	As an Employee, I want to track my requests.	The request list returns only records whose requester_id is the current user.
US04	As an Admin/Manager, I want to approve or reject PENDING requests.	Only PENDING requests can be reviewed; rejection requires a note; reviewer and timestamp are stored.
US05	As Warehouse Staff, I want to issue an approved request.	Only APPROVED requests are issuable; every item must have sufficient stock; success stores transaction/detail rows, decreases inventory and marks request ISSUED.
US06	As Warehouse Staff, I want to receive stock.	A multi-item IN transaction increases each inventory quantity and stores a transaction record.
US07	As Warehouse Staff, I want to prevent over-issue.	If any OUT/ISSUE quantity exceeds current stock, the entire database transaction rolls back.
US08	As an Admin/Manager, I want to manage products and suppliers.	Create/update data persists; deactivate is a soft state change; restore reactivates the record.
US09	As an operational user, I want to find inventory quickly.	Search/filter/sort return database-derived results and low-stock status is calculated from quantity <= minimum_stock.
US10	As an Admin/Manager, I want a transaction report.	The report supports type/date filters and returns transaction rows plus summary counts.

7. Project Management, Project Board and Team Contribution

The project should be managed through a GitHub repository created during the examination period with meaningful, progressive commits. The final repository must not appear as a single bulk upload. Each technical change should be reviewed before being committed.

Member	Responsibility / Evidence Ownership
Đặng - Lead	Requirements, scope, architecture, repository coordination, deployment coordination, integration and final report.
Trung	PHP backend and database review; SQL schema; data integrity; API/database evidence.
Huyền	Frontend/UI/UX review; responsive layout; screenshots and design-workflow evidence.
Duy	Business workflow verification; Use Case, Activity and Sequence diagrams; end-to-end demo flow.
Vinh	Testing, security review, defect tracking, production test evidence and final test report.

7.1 Project Board

[TO BE UPDATED - PROJECT BOARD LINK]

Insert the actual GitHub Project Board or approved project-board link and capture a screenshot showing task status and ownership.

7.2 Progressive Commit Plan

1. Initial production project structure
2. Refactor PHP backend into JSON API
3. Add database-backed authentication sessions
4. Implement Product and Supplier CRUD
5. Implement inventory Stock In / Stock Out
6. Implement Employee request
7. Implement Manager approval/rejection
8. Implement Warehouse issue + atomic inventory update
9. Add Vercel-ready frontend and API integration
10. Add search/filter/sort/reporting
11. Configure backend Docker deployment
12. Configure Vercel build
13. Add production test plan
14. Fix production defects
15. Update README and final report evidence

8. System Diagrams

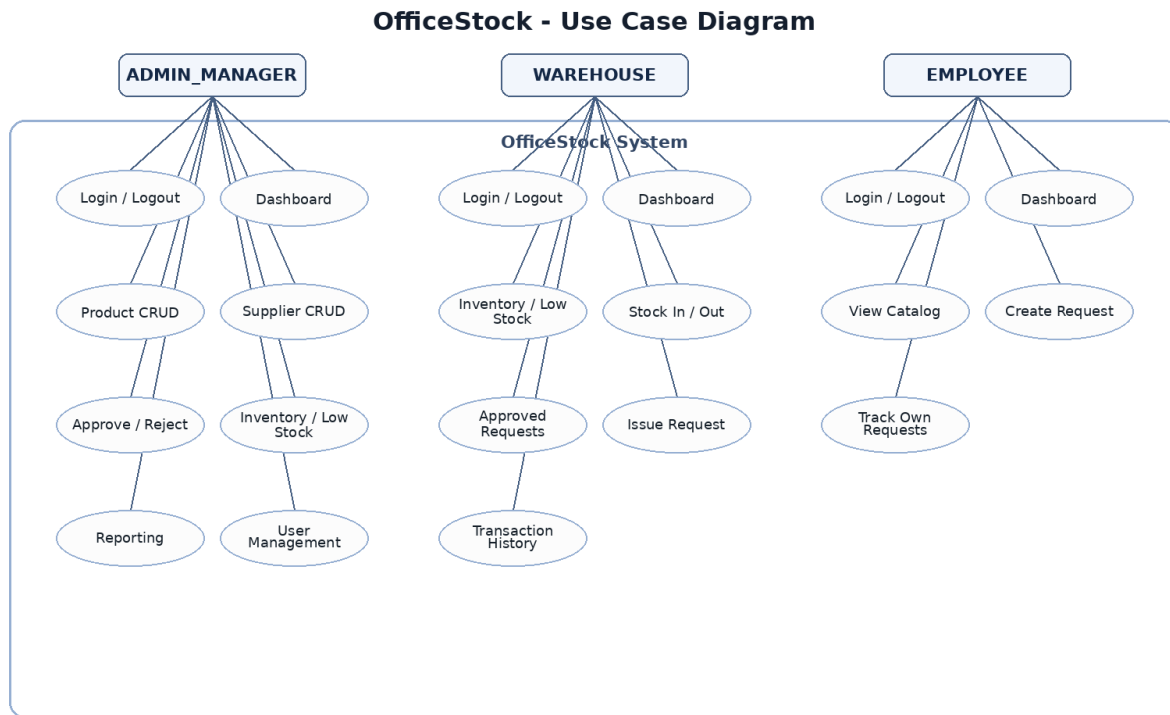


Figure 1. Use Case Diagram - exactly three actors and implemented OfficeStock functions.

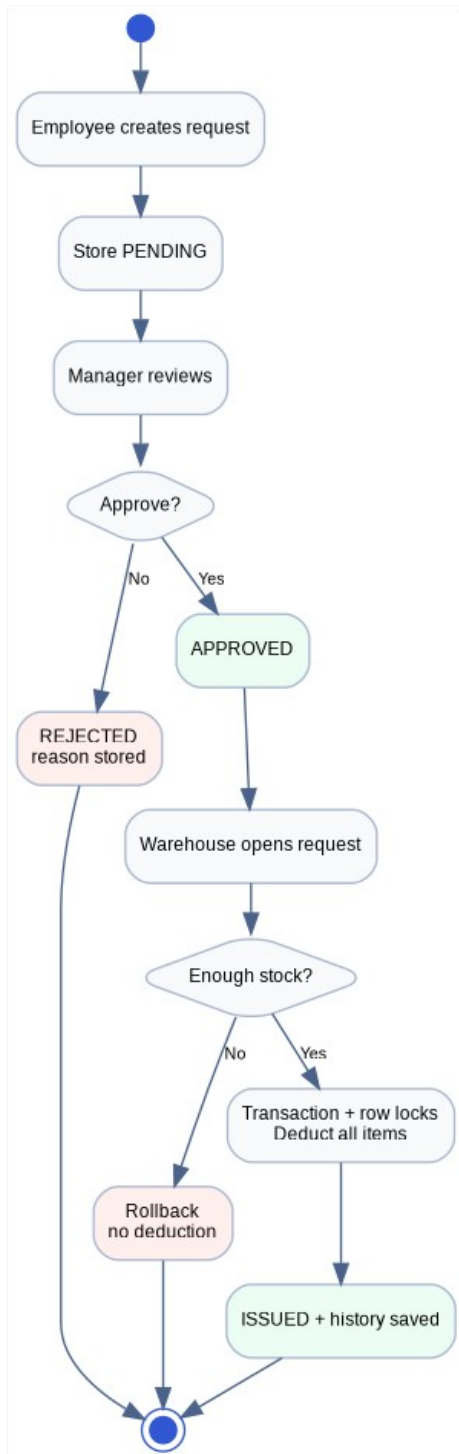


Figure 2. Activity Diagram - request, approval, stock validation, issue and rollback path.

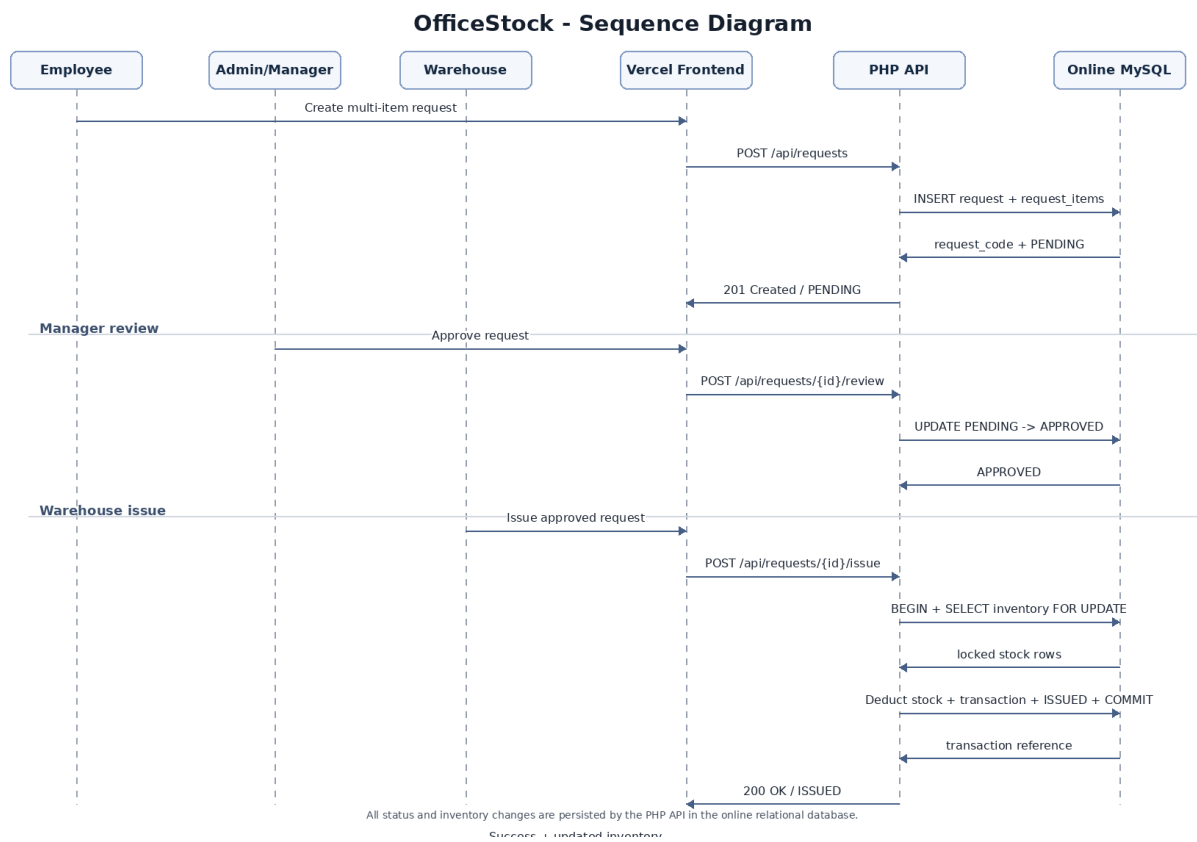


Figure 3. Sequence Diagram - Vercel frontend calls PHP API, which persists workflow changes in the online database.

The diagrams are based on the refactored source package. Before submission, the team must re-check them against any production code change made during deployment debugging.

9. Application Architecture and Technology Stack

9.1 Production Architecture

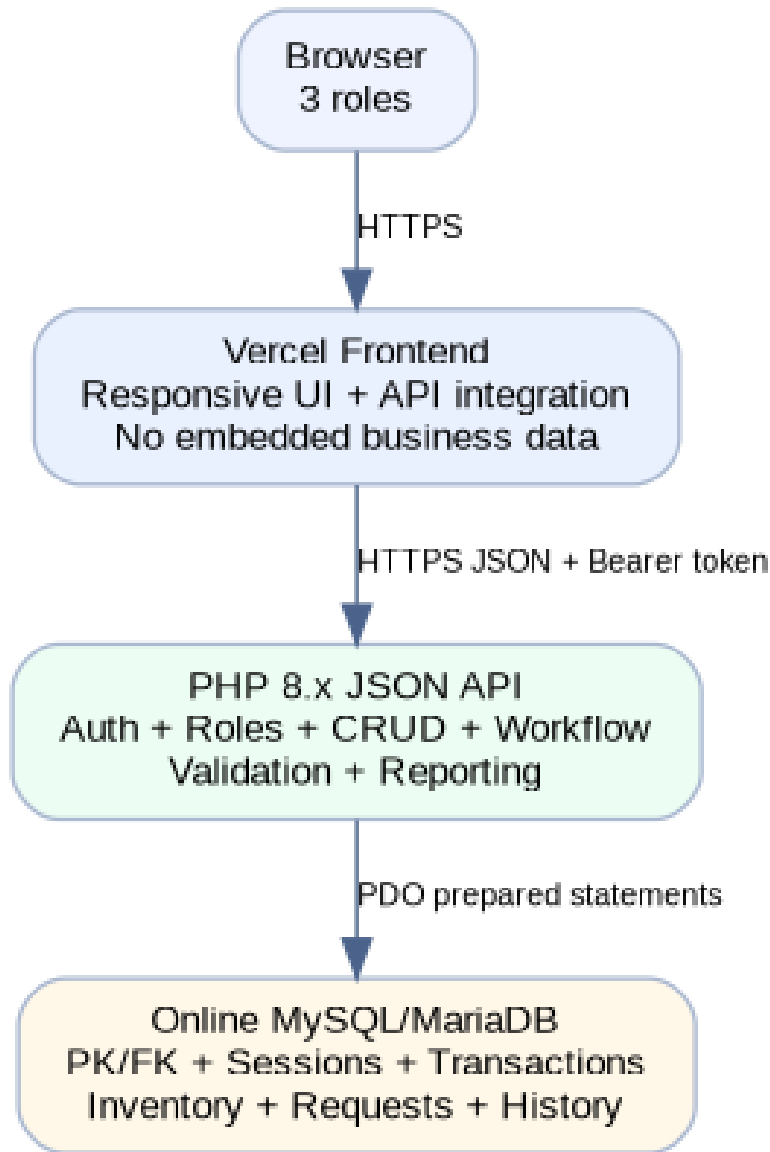


Figure 4. OfficeStock production application architecture.

The frontend and backend are deployed independently. The frontend is a responsive browser application prepared for Vercel. It calls the PHP API over HTTPS. The PHP backend owns authentication, authorization, validation, CRUD, workflow and reporting. PDO connects to the online relational database using hosting environment variables.

9.2 Deployment Architecture

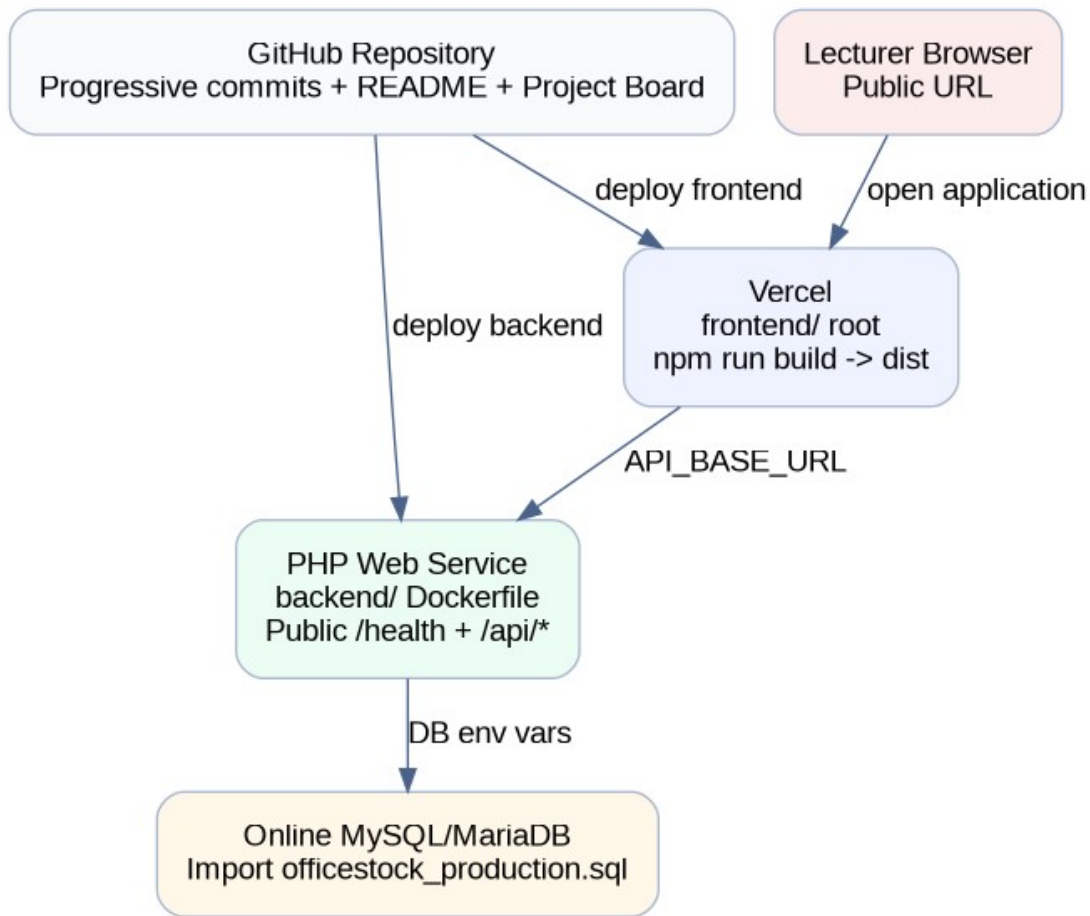


Figure 5. Deployment architecture and repository-to-hosting flow.

9.3 Technology Stack

Layer	Technology	Purpose
Frontend	HTML5 / CSS3 / ES Modules	Responsive business UI, forms, navigation and API integration.
Frontend Build	Node.js built-in build script	Generates static dist and injects API_BASE_URL without third-party runtime packages.
Frontend Hosting	Vercel	Public frontend URL.
Backend	PHP 8.x	Authentication, authorization, CRUD, workflow, validation and reporting.
Database Access	PDO / pdo_mysql	Prepared statements and database transactions.
Backend Hosting	Docker-compatible PHP web service	Public PHP API and /health endpoint.
Database	Online MySQL/MariaDB	Relational master/transaction data and application sessions.
Version Control	GitHub	Progressive commits, README and Project Board.

10. Database Schema and Data Dictionary

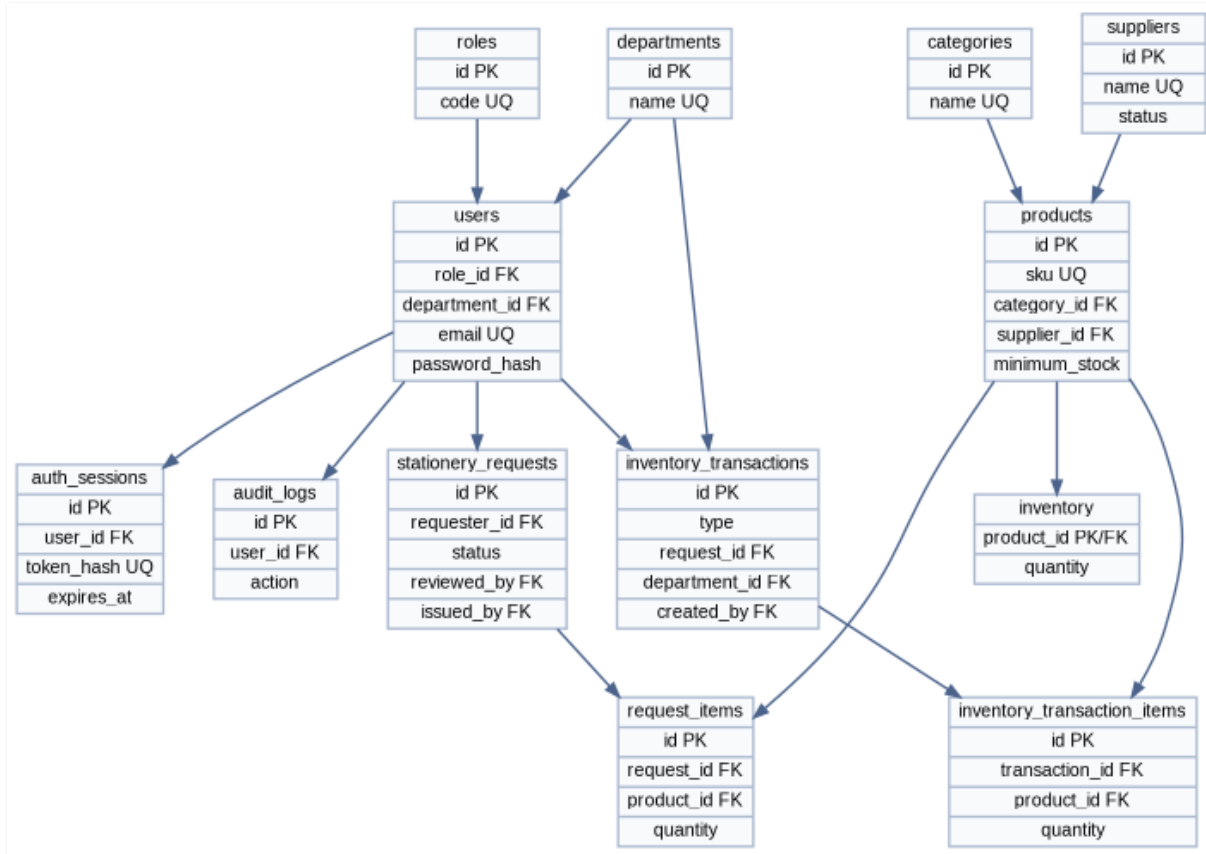


Figure 6. OfficeStock logical ERD based on backend/database/officestock_production.sql.

The database separates master data, current inventory, request workflow, inventory transaction history, application sessions and audit evidence. The schema includes primary keys, foreign keys, unique constraints, check constraints and indexes relevant to workflow queries.

Table	Field	Type	Definition
roles	id	INT PK	Role identifier.
roles	code	VARCHAR UNIQUE	ADMIN_MANAGER / WAREHOUSE / EMPLOYEE.
departments	id	INT PK	Department identifier.
departments	name	VARCHAR UNIQUE	Department name.
users	id	INT PK	User identifier.
users	email	VARCHAR UNIQUE	Login identifier.
users	password_hash	VARCHAR	Hashed password; plaintext is never stored.
users	role_id	INT FK	References roles.
users	department_id	INT FK NULL	Employee department.
users	status	ENUM	ACTIVE or LOCKED.
auth_sessions	id	BIGINT PK	Application session identifier.
auth_sessions	user_id	INT FK	Session owner.
auth_sessions	token_hash	CHAR(64) UNIQUE	SHA-256 hash of random bearer session token.
auth_sessions	expires_at	DATETIME	Server-side session expiry.

categories	id	INT PK	Product category identifier.
categories	name	VARCHAR UNIQUE	Category name.
suppliers	id	INT PK	Supplier identifier.
suppliers	name	VARCHAR UNIQUE	Supplier name.
suppliers	status	ENUM	ACTIVE/INACTIVE soft state.
products	id	INT PK	Product identifier.
products	sku	VARCHAR UNIQUE	Product SKU.
products	category_id	INT FK	Product category.
products	supplier_id	INT FK NULL	Preferred supplier.
products	minimum_stock	INT	Low-stock threshold.
products	unit_cost	DECIMAL	Reference unit cost.
products	status	ENUM	ACTIVE/INACTIVE soft state.
inventory	product_id	INT PK/FK	One current-stock row per product.
inventory	quantity	INT	Current non-negative quantity.
stationery_requests	id	INT PK	Request identifier.
stationery_requests	request_code	VARCHAR UNIQUE	Human-readable request code.
stationery_requests	requester_id	INT FK	Employee who submitted.
stationery_requests	status	ENUM	PENDING/APPROVED/ REJECTED/ISSUED/ CANCELLED.
stationery_requests	reviewed_by	INT FK NULL	Manager reviewer.
stationery_requests	issued_by	INT FK NULL	Warehouse issuer.
request_items	id	INT PK	Request line identifier.
request_items	request_id	INT FK	Request header.
request_items	product_id	INT FK	Requested product.
request_items	quantity	INT > 0	Requested quantity.
inventory_transactions	id	INT PK	Transaction header identifier.
inventory_transactions	reference_code	VARCHAR UNIQUE	Transaction reference.
inventory_transactions	type	ENUM	IN / OUT / REQUEST_ISSUE.
inventory_transactions	request_id	INT FK NULL	Related request when issued.
inventory_transactions	created_by	INT FK	Warehouse user who executed transaction.
inventory_transaction_items	transaction_id	INT FK	Transaction header.
inventory_transaction_items	product_id	INT FK	Transaction product.
inventory_transaction_items	quantity	INT > 0	Moved quantity.
audit_logs	id	BIGINT PK	Audit row identifier.
audit_logs	user_id	INT FK NULL	Actor where available.
audit_logs	action	VARCHAR	LOGIN, CREATE_REQUEST, ISSUE_REQUEST, etc.

11. Implementation Evidence

Module	Implemented Behavior	Source Evidence
Authentication	Login, application session token, logout revocation, current user.	backend/app/Core/Auth.php; backend/public/index.php
Authorization	Role checks on privileged services/endpoints.	Auth::requireRole calls in Services and router.
Product CRUD	Create/read/update/deactivate/restore + search/sort.	CatalogService.php + /api/products endpoints.
Supplier CRUD	Create/read/update/deactivate/restore + search/filter.	CatalogService.php + /api/suppliers endpoints.
Inventory	Current stock, low stock, Stock In/Out, negative-stock prevention.	InventoryService.php
Employee Request	Multi-item PENDING request and own-request restriction.	RequestService::create/list/cancelOwn
Manager Review	Approve/reject only PENDING; rejection reason required.	RequestService::review
Warehouse Issue	APPROVED-only; transaction, locks, deduction, ISSUED.	RequestService::issue
Reporting	Transaction rows with type/date filters and summary.	InventoryService::transactions/ reportSummary
Frontend	Role navigation, forms, status feedback, API integration.	frontend/src/js/app.js; styles.css

11.1 Production Screenshot Evidence

[TO BE UPDATED - PUBLIC VERCEL URL]

Browser address bar and working OfficeStock login page.

[TO BE UPDATED - EMPLOYEE DASHBOARD]

Authenticated Employee dashboard loaded from production database.

[TO BE UPDATED - EMPLOYEE REQUEST]

Submitted PENDING request with 10 pens + 5 notebooks.

[TO BE UPDATED - MANAGER APPROVAL]

Manager request queue and approval evidence.

[TO BE UPDATED - WAREHOUSE ISSUE]

Approved request issued successfully.

[TO BE UPDATED - INVENTORY UPDATED]

Inventory quantities after issue.

[TO BE UPDATED - TRANSACTION HISTORY]

REQUEST_ISSUE transaction and product lines.

[TO BE UPDATED - LOW STOCK]

Low-stock badge/filter evidence.

[TO BE UPDATED - REPORTING]

Manager report filters and transaction summary chart.

[TO BE UPDATED - RESPONSIVE VIEW]

Mobile-width production frontend.

12. UI/UX Design Workflow

Step	Design Decision	Evidence
1. Role tasks	Start from the actions each role must perform.	ROLE_PERMISSION_MATRIX.md
2. Navigation	Show only modules relevant to the signed-in role.	frontend/src/js/app.js
3. Information hierarchy	Dashboard cards surface product, inventory, low-stock and request metrics.	Dashboard UI
4. Form design	Labels, required fields, numeric minimums and clear action buttons.	Product/request/stock forms
5. Status feedback	PENDING/APPROVED/REJECTED/ISSUED and LOW/NORMAL badges.	Frontend status badges
6. Error handling	API errors are surfaced as toast/error panels.	api.js + ui.js
7. Responsiveness	Sidebar collapses and grids adapt to tablet/mobile widths.	styles.css media queries
8. Evidence capture	Production desktop/mobile screenshots inserted before submission.	Final report screenshots

The frontend deliberately uses no static business dataset. Interface tables, dashboard metrics, requests, inventory and reports are rendered from API responses.

13. Authentication and Authorization

13.1 Login and Application Session

- Frontend sends email/password to POST /api/auth/login over HTTPS.
- PHP looks up the account using a prepared statement and verifies the password hash.
- PHP generates a cryptographically random session token and stores only its SHA-256 hash in auth_sessions.
- Frontend stores the returned token in sessionStorage and sends it in the Authorization header.
- Protected endpoints resolve the current user from the database session row and enforce expiry.
- Logout deletes the server-side session row, making the token unusable.

13.2 Role Authorization

Frontend navigation improves usability but is not treated as a security boundary. Server-side service methods call `Auth::requireRole` before privileged business operations. An Employee cannot approve requests by directly calling the manager endpoint, and a Manager cannot issue warehouse requests unless the account has the WAREHOUSE role.

14. Core Business Workflow

The central workflow is the most important implementation path for demonstration. It is fully represented in the PHP backend and relational database model.

22. Employee creates a request with a reason and multiple unique product items.
23. The backend inserts stationery_requests with status PENDING and request_items inside one transaction.
24. Admin/Manager reads PENDING requests and selects Approve or Reject.
25. Approve changes status to APPROVED and records reviewer/time. Reject changes status to REJECTED and requires a reason.
26. Warehouse sees APPROVED requests and selects Issue.
27. The backend locks the request and inventory rows using `SELECT ... FOR UPDATE`.
28. Every requested quantity is validated against current stock.
29. If any item is insufficient, the transaction rolls back and no quantity is deducted.
30. If every item is available, the backend inserts a REQUEST_ISSUE transaction and detail rows, decreases inventory, updates request to ISSUED and commits.
31. Inventory, transaction history and dashboard/report metrics then reflect the database changes.

15. Search, Filter, Sort and Reporting

Function	Implementation
Product Search	SKU/name text query.
Product Sort	Allow-listed sort columns including name, SKU, quantity and minimum stock.
Supplier Search/Filter	Name/email/phone search and ACTIVE/INACTIVE filter.
Inventory Search	SKU/name search.
Inventory Filter	Low stock or normal stock.
Inventory Sort	Name/SKU/quantity/minimum with allow-listed SQL expressions.
Request Filter	Status-based request list, additionally constrained by role.
Transaction Report	Type/date filters and summary count by transaction type.
Report Visualization	Frontend transaction-volume bar chart generated from API summary.

16. Validation and Error Handling

- Required strings are trimmed and maximum lengths are checked.
- Email addresses use server-side format validation.
- Product IDs and quantities must be positive integers.
- Minimum stock and unit cost must be non-negative.
- Duplicate products are rejected inside one request/transaction payload.
- Rejecting a request requires a rejection reason.

- Only known status/sort values are accepted through allow-lists.
- Over-issue raises a business-rule error and rolls back the database transaction.
- Frontend forms add usability validation, while the PHP backend remains authoritative.
- API errors use structured JSON codes such as `VALIDATION_ERROR`, `FORBIDDEN` and `BUSINESS_RULE_ERROR`.

17. Security Controls

Control	Implementation / Evidence
Password protection	<code>password_hash()</code> for seed/runtime user passwords and <code>password_verify()</code> during login.
Prepared statements	PDO prepared statements; emulated prepares disabled.
Application-session protection	Random 256-bit token; hash stored server-side; expiry; revocation on logout.
Authorization	Backend <code>Auth::requireRole</code> checks.
Input validation	Validator allow-lists and numeric/string validation.
Inventory integrity	Transactions + <code>SELECT FOR UPDATE</code> ; rollback on failure.
CORS	Exact allowed frontend origin(s) from <code>CORS_ALLOWED_ORIGINS</code> .
Secrets	Database/API production settings use environment variables; <code>.env</code> ignored.
Error disclosure	Production returns generic unexpected-server error unless <code>APP_DEBUG</code> explicitly enabled.
Security headers	<code>nosniff</code> , <code>DENY</code> framing, <code>referrer</code> and <code>permissions</code> headers.

Cookie-based CSRF is not used as the authentication mechanism in the separated-domain production architecture. The session credential is explicitly attached to requests as a bearer authorization header, so conventional ambient-cookie CSRF does not apply. The production frontend origin is still restricted by the backend CORS allow-list.

18. Testing, Results and Unresolved Defects

Static verification has been performed on the generated production source package. Runtime production tests remain NOT RUN until the team deploys real public resources. This distinction prevents fabricated PASS results.

Verification	Result
PHP syntax lint for all backend PHP files	PASS
JavaScript syntax check for frontend modules	PASS
Frontend production build with <code>API_BASE_URL</code>	PASS
Exactly three seeded roles by SQL inspection	PASS
Workflow transaction/locking logic by source inspection	PASS
Public frontend URL	NOT RUN
Public PHP API database connectivity	NOT RUN
TC01-TC20 end-to-end production execution	NOT RUN

ID	Test	Expected	Actual	Status
TC01	Login Admin/Manager	Manager dashboard opens	[TO BE UPDATED]	NOT RUN

TC02	Login Warehouse	Warehouse dashboard opens	[TO BE UPDATED]	NOT RUN
TC03	Login Employee	Employee dashboard opens	[TO BE UPDATED]	NOT RUN
TC04	Wrong password	Login rejected safely	[TO BE UPDATED]	NOT RUN
TC05	Role authorization	Forbidden operation returns 403	[TO BE UPDATED]	NOT RUN
TC06	Product CRUD	Create/update/deactivate/restore persists	[TO BE UPDATED]	NOT RUN
TC07	Supplier CRUD	Create/update/deactivate/restore persists	[TO BE UPDATED]	NOT RUN
TC08	Search/filter/sort	Results match query	[TO BE UPDATED]	NOT RUN
TC09	Employee create request	PENDING request + two items stored	[TO BE UPDATED]	NOT RUN
TC10	Manager approve	PENDING -> APPROVED	[TO BE UPDATED]	NOT RUN
TC11	Manager reject	REJECTED + reason stored	[TO BE UPDATED]	NOT RUN
TC12	Warehouse issue	ISSUED + transaction saved	[TO BE UPDATED]	NOT RUN
TC13	Inventory update	Stock decreases exactly	[TO BE UPDATED]	NOT RUN
TC14	Over-issue rejection	Rollback/no partial deduction	[TO BE UPDATED]	NOT RUN
TC15	Low-stock alert	LOW state displayed	[TO BE UPDATED]	NOT RUN
TC16	Dashboard	Metrics match production DB	[TO BE UPDATED]	NOT RUN
TC17	Reporting	Type/date report works	[TO BE UPDATED]	NOT RUN
TC18	Logout	Old token returns 401	[TO BE UPDATED]	NOT RUN
TC19	Validation	Invalid input is rejected	[TO BE UPDATED]	NOT RUN
TC20	Security	Secrets/CORS/auth/SQL controls verified	[TO BE UPDATED]	NOT RUN

18.1 Unresolved Defects

[TO BE UPDATED - DEFECT LOG]

After production tests, list only real unresolved defects with severity, reproduction steps, owner and workaround.

19. Deployment and Setup Guide

19.1 Production Deployment

32. Create/finalize the GitHub repository and push meaningful progressive commits.
33. Provision an online MySQL/MariaDB database and import backend/database/officestock_production.sql.
34. Create a Docker PHP web service using backend/Dockerfile.
35. Set backend environment variables: application mode, allowed frontend origin and database credentials.
36. Deploy the backend and verify /health reports the database as connected.
37. Create a Vercel project using frontend/ as the root directory.
38. Set API_BASE_URL to the public PHP API origin.
39. Build with npm run build and deploy dist.
40. Update backend CORS_ALLOWED_ORIGINS to the exact final Vercel origin and redeploy if necessary.
41. Run the full production test plan through the public frontend.

19.2 Developer Verification Without Production Secrets

The source package can be syntax-checked and the frontend can be built without exposing production credentials. Database-dependent behavior requires a compatible MySQL/MariaDB database. Production credentials must remain in hosting environment variables.

19.3 Environment Variables

Component	Variable	Purpose
Backend	APP_ENV	Production mode.
Backend	APP_DEBUG	Keep false for production.
Backend	APP_URL	Public PHP API URL.
Backend	CORS_ALLOWED_ORIGINS	Exact Vercel origin(s).
Backend	SESSION_TTL_HOURS	Application-session expiry.
Backend	DB_HOST / DB_PORT / DB_NAME / DB_USER / DB_PASSWORD	Online database connection.
Frontend	API_BASE_URL	Public PHP API base URL used by build script.

20. Test Accounts and Demo Scenario

Role	Email	Assessment Password
ADMIN_MANAGER	manager@officestock.demo	Manager@2026
WAREHOUSE	warehouse@officestock.demo	Warehouse@2026
EMPLOYEE	employee@officestock.demo	Employee@2026

These passwords are assessment-only sample credentials. The SQL stores password hashes, not plaintext passwords.

20.1 Recommended Demonstration

42. Open the public Vercel frontend.
43. Login as Employee.
44. Create a request for 10 Blue Ballpoint Pens and 5 A5 Notebooks.
45. Verify the request is PENDING.
46. Login as Admin/Manager and approve the request.
47. Login as Warehouse and issue the APPROVED request.
48. Open Inventory and verify the two quantities decreased.
49. Open Transaction History and verify the REQUEST_ISSUE reference.
50. Login as Admin/Manager and show updated dashboard/report values.
51. Show a negative test: attempt an unauthorized action or over-issue and demonstrate safe rejection.

21. Requirement-to-Evidence Traceability Matrix

Exam Area	OfficeStock Implementation	Required Evidence
Project management + requirements	Requirements, user stories, Project Board, member roles	docs/REQUIREMENTS.md; PROJECT_BOARD.csv; GitHub evidence
Use Case / Activity / Sequence	Diagrams based on implemented	docs/diagrams/*.png

	roles/workflow	
UI/UX + responsive	Role navigation, forms, badges, errors, media queries	frontend/src + production screenshots
Database + PHP connection	MySQL schema and PDO connection	backend/database + Database.php
Authentication + roles	Hashed passwords, application sessions, exactly 3 roles	Auth.php + roles seed
CRUD	Product/Supplier endpoints and UI	CatalogService.php + frontend
Complete business workflow	Request -> Approval -> Issue -> Inventory update	RequestService.php + DB evidence
Search/filter/sort/report	Product/inventory filters and transaction reports	CatalogService/InventoryService + frontend
Security	Prepared statements, authorization, session, CORS, env secrets	backend Core/Services
Testing/results	TC01-TC20 + actual production results	PRODUCTION_TEST_PLAN.md
Packaging/setup	README, Dockerfile, Vercel config, SQL	repository package
Live demo	Public frontend/backend/database	[TO BE UPDATED - REAL URLs]

22. Third-party Assets and Licenses

The production frontend is intentionally dependency-light. It uses browser-standard HTML, CSS and JavaScript modules and does not require a third-party UI template. The backend uses the official PHP runtime image for container deployment and built-in PHP/PDO functionality. Graphviz was used during documentation generation to render diagrams. The final repository README should list any additional library, icon, image or template added by the team after this report is generated.

[TO BE UPDATED - THIRD-PARTY ASSET LIST]

Add the exact source/license of any asset or library introduced during final UI/deployment changes.

23. AI / Tool-use Declaration

Field	Declaration
AI/tool name(s)	[TEAM TO CONFIRM ACTUAL AI/TOOLS USED]
Purpose of use	Architecture refactoring, code drafting, validation review, documentation, test-plan drafting and report preparation.
Main parts assisted	PHP API structure, frontend/API integration, database review, deployment configuration, diagrams and report drafting.
How output was verified	PHP syntax lint, JavaScript syntax checks, frontend build, team review, production tests after deployment, and ability to explain/modify the work.

The team remains responsible for correctness, security, licensing, originality and the ability to explain or modify the submitted work during the demonstration/viva.

24. Limitations and Future Development

24.1 Current Limitations

- One organization / one logical warehouse.
- No automatic procurement workflow.
- No MFA or enterprise SSO.
- No email/SMS notification integration.
- No native mobile application.
- Public monitoring/backups depend on the selected hosting/database provider.

24.2 Future Development

- Multiple warehouses and transfer transactions.
- Purchase-order and replenishment workflow.
- Department budget/spending limits.
- Notification service for approvals and low-stock events.
- Report export and richer analysis.
- Demand forecasting only after sufficient historical data is available.

25. Viva / Demo Preparation

Question	Short Answer
Why is the PHP backend retained?	The examination explicitly requires PHP connection/backend evidence; PHP also owns the real business rules and database access.
How many roles are implemented?	Exactly three: ADMIN_MANAGER, WAREHOUSE and EMPLOYEE.
Where are passwords stored?	Only password hashes are stored in users.password_hash.
How is SQL injection mitigated?	PDO prepared statements are used and emulated prepares are disabled.
How is a logged-in session protected?	A random token is issued; only its SHA-256 hash is stored server-side with expiry and revocation.
Why can an Employee not approve a request?	The PHP review service requires ADMIN_MANAGER before updating a request.
How can one request contain multiple products?	stationery_requests is the header and request_items contains product/quantity detail rows.
What happens after approval?	Status becomes APPROVED; Warehouse can then see and issue it.
How do you prevent negative stock?	Inventory rows are locked inside a DB transaction and quantities are validated before update.
What if one item is insufficient?	The entire issue transaction rolls back; no partial stock deduction is committed.
Where is issue history stored?	inventory_transactions and inventory_transaction_items.
How is low stock detected?	quantity <= minimum_stock.
How is Employee data privacy enforced?	Request queries add requester_id=current authenticated user.
Where is validation performed?	Both frontend usability validation and authoritative PHP Validator/business rules.
Why is conventional cookie CSRF not the main control?	The separated frontend uses an explicit bearer authorization header rather than an ambient authentication cookie.
What does CORS do?	The backend accepts browser API requests only from configured production frontend origins.
What is the main deployment architecture?	Vercel frontend -> PHP API -> online MySQL/MariaDB.
What results have actually been verified now?	Static PHP/JS/build checks; production URL/database workflow tests

	remain NOT RUN until hosting is connected.
Why are some report fields placeholders?	The report does not invent URLs, screenshots or PASS results without real production evidence.
What is the most important demo flow?	Employee Request -> Manager Approve -> Warehouse Issue -> Inventory Update -> Transaction History.

26. Final Submission Checklist

- ☐ Application starts successfully from the public frontend instructions.
- ☐ Online database contains the submitted schema and sufficient sample data.
- ☐ GitHub repository has visible progressive history.
- ☐ README includes purpose, features, technology, deployment, database, test accounts and limitations.
- ☐ All public links are accessible to the lecturer.
- ☐ Exactly three role accounts work.
- ☐ Employee -> Manager -> Warehouse workflow works on production.
- ☐ Report and diagrams match implemented source/database.
- ☐ Production screenshots are inserted and annotated.
- ☐ TC01-TC20 have real Actual Result and PASS/FAIL values.
- ☐ No production password, private API key or personal data is committed.
- ☐ Third-party assets/libraries are disclosed.
- ☐ AI/tool-use declaration reflects actual team use.
- ☐ File names are changed to the lecturer-required naming format.

Final Evidence Status

Source refactoring and static verification are complete in the generated package. Public deployment, public URLs, production screenshots and end-to-end runtime results must be completed by the team using real hosting/database accounts before this report is treated as submission-final.